

ClanWorld v4.5 — Keeper Integration Spec

Status: Draft **Purpose:** Define how the ClanWorld game engine's `heartbeat()` function is driven in production demos for two parallel hackathon submissions, and how each tick triggers a downstream backend update. **Audience:** Project manager, agent/backend implementers, contract implementers, demo operators. **Scope:** Out-of-process orchestration only. Contract semantics are unchanged from v4 / v4.1 / v4.2 / v4.3.

0. Summary

ClanWorld's world clock advances when an external caller invokes `heartbeat()`. Per `IClanWorld.sol`, this entrypoint is **permissionless, takes no caller-supplied randomness**, and is **rate-limited internally** by `WorldState.nextHeartbeatAtTs`. Therefore any reliable scheduled invocation source is acceptable — duplicate calls inside the same window revert cheaply and do not corrupt state.

This spec locks two parallel keeper integrations:

1. **KeeperHub on Base Sepolia** — primary integration for the KeeperHub hackathon submission.
2. **Foundry script on World Chain Sepolia** — secondary integration for the World mini app hackathon submission.

Both paths drive the same contract code with the same calldata. They run as **independent realm deployments** on independent chains.

1. Locked decisions

1.1 Tick cadence

- Target: **60 seconds per tick**.
- This matches `ClanWorldConstants.HEARTBEAT_INTERVAL_SECONDS = 60` and v4.1 §A1.
- Cadence is **configurable** by changing the keeper's schedule expression. The contract's internal floor (`nextHeartbeatAtTs`) protects against over-firing regardless of keeper config.

1.2 Heartbeat endpoint

- Function: `heartbeat()` on the deployed `IClanWorld` contract.
- Arguments: none.
- Caller: any funded EOA. No allowlist.
- Failure mode: revert if called too early. This is acceptable and intentional — the keeper does not need to coordinate.

1.3 Two-realm model

- **Realm A:** deployed on Base Sepolia. Driven by KeeperHub.
- **Realm B:** deployed on World Chain Sepolia. Driven by a local Foundry loop.
- The two realms share **no onchain state**. Each is its own `ClanWorld` season instance.
- This decouples the two hackathon submissions and removes any cross-chain coordination requirement.

1.4 Post-tick webhook

- After each successful `heartbeat()` tx, the keeper fires a webhook to the project backend.
 - Purpose: prompt the backend / indexer to read fresh state, push UI updates, and notify any agent harnesses that a new tick is open.
 - The webhook is **out-of-band signaling only**. It is not authoritative game state. The contract remains the source of truth.
-

2. Primary path — KeeperHub on Base Sepolia

2.1 Deployment surface

- Chain: Base Sepolia.
- Keeper: a single KeeperHub workflow per realm.
- Wallet: KeeperHub-managed Turnkey wallet, funded with a small amount of Base Sepolia ETH for gas.

2.2 Workflow shape

The workflow has three nodes:

1. **Trigger:** cron, expression `* * * * *` (every minute = the 60s tick cadence).
2. **Action:** Web3 contract write calling `heartbeat()` on the realm's deployed engine address. No args.
3. **Action:** HTTP POST to the project backend webhook URL.

Order: trigger → contract write → webhook fire. The webhook fire is the last step and uses the prior step's tx hash as part of its payload.

2.3 Cadence configurability

Cadence is changed by editing the cron expression on the workflow. There is no need to redeploy the contract. Cron's 1-minute granularity is the floor; sub-minute ticks are out of scope for v1.

2.4 Failure handling

- If the contract write reverts (e.g. heartbeat called too early due to clock skew), the workflow may still fire the webhook. Backend should treat webhook events as advisory and reconcile against onchain state.
- KeeperHub's built-in retry / nonce / gas handling is sufficient for v1. No custom retry logic is required.

2.5 Why this path

KeeperHub is the keeper integration the project is submitting against for the KeeperHub hackathon. Using it on Base Sepolia keeps the integration simple, fully managed, and demoable from KeeperHub's own UI.

3. Secondary path — Foundry on World Chain Sepolia

3.1 Deployment surface

- Chain: World Chain Sepolia (chain ID 4801).
- Keeper: a Foundry-driven loop running on a developer machine, CI runner, or small VPS during the demo window.
- Wallet: any funded EOA with World Chain Sepolia ETH.

3.2 Loop shape

The simplest acceptable form is a shell loop calling `cast send` (or `forge script --broadcast`) once per tick:

```
while true; do
  cast send "$ENGINE_ADDR" "heartbeat()" \
    --rpc-url "$WORLDCHAIN_SEPOLIA_RPC" \
    --private-key "$KEEPER_PK"
  curl -X POST "$BACKEND_WEBHOOK_URL" -d '{"chain":"worldchain-sepolia"}'
  sleep 60
done
```

A `forge script` variant that wraps the same call is also acceptable if a Solidity-side tx description is preferred.

3.3 Cadence configurability

Cadence is changed by editing the `sleep` value or wrapping the loop in `cron` / `systemd-timer`. Same 60s default as the primary path.

3.4 Why this path

This path exists because the World mini app hackathon requires the game to run on World Chain. KeeperHub does not list World Chain in its published supported-chain set as of this writing, so a self-hosted Foundry loop is the simplest reliable substitute. The contract surface is identical, so no engine code changes.

4. Webhook contract (soft)

The following is a **soft decision** — the project manager and backend/agent implementers should finalize the exact shape based on backend needs.

4.1 Suggested minimum payload

```
{
  "chain": "base-sepolia" | "worldchain-sepolia",
  "engineAddress": "0x...",
  "txHash": "0x...",
  "firedAtTs": 1735689600,
  "source": "keeperhub" | "foundry-loop"
}
```

4.2 Open items left to the PM / implementers

- Authentication on the webhook (shared secret? signed payload? IP allowlist?).
- Whether to include the post-tick `currentTick` in the payload (requires an extra RPC read in the keeper) or have the backend re-derive it from the chain.
- Whether to fire the webhook on revert as well as success.
- Backend endpoint URL, hosting, and rate-limit handling.

These are intentionally not locked here. The contract layer does not depend on any of them.

5. Operational notes

5.1 Permissionless heartbeat is the safety net

Because `heartbeat()` is permissionless and self-rate-limiting:

- A backup keeper (manual cast send, second KeeperHub workflow, etc.) can be run alongside the primary keeper without coordination.
- Running both keepers simultaneously on the same realm is safe — only one tx per window will succeed.
- If the primary keeper goes down mid-demo, anyone watching the demo can keep the world ticking.

This is an intentional design property of the engine and should be highlighted in both hackathon submissions.

5.2 Funding

- Base Sepolia: KeeperHub Turnkey wallet must hold Base Sepolia ETH.
- World Chain Sepolia: Foundry keeper EOA must hold World Chain Sepolia ETH.
- Funding levels for a demo-length season (~360 ticks at 60s per tick = ~6 hours) are expected to be trivial. Exact gas budgeting is not specified here.

5.3 Independence from agents

The keeper runs unconditionally. It does **not** wait for Elder agent activity. Agent-driven `submitClanOrders` calls happen asynchronously on top of the heartbeat; the heartbeat closes ticks regardless of whether any orders landed in that tick. This matches v4 §2

semantics.

6. Out of scope for v4.5

The following are explicitly not addressed in this spec and remain open for the PM and implementers:

- Multi-realm orchestration on a single chain.
- Keeper failover automation.
- Backend webhook schema and auth.
- Indexer architecture beyond “the webhook prompts the backend to update.”
- Production deployment to either chain’s mainnet.
- Agent harness deployment topology.
- Mini app frontend integration with the World mini app SDK / MiniKit. (This is part of the World hackathon submission but does not affect the keeper integration.)

These are deliberately left flexible so individual workstream owners can integrate their bounty requirements without renegotiating the keeper layer.

7. Locked summary

- 60s default tick cadence, configurable via keeper schedule expression.
- `heartbeat()` is the only endpoint the keeper calls.
- KeeperHub on Base Sepolia is the primary path.
- Foundry shell loop on World Chain Sepolia is the secondary path.
- Each chain runs an independent realm; no shared onchain state.
- Each successful tick fires a webhook to the project backend; payload shape is soft.
- Permissionless heartbeat means backup keepers are always safe to run.